

image_name.png

Research Report: ROS2 NAV2 Package and Architecture

Topic: Theoretical Background and Practical Implementation of NAV2

April 29, 2026

Contents

1	Theoretical Background of NAV2	4
1.1	Overview of the Navigation Stack (NAV2)	4
1.2	Map and Localization	4
1.3	Costmap	4
1.4	Path Planning	4
1.5	Recovery Behaviors	5
1.6	Behavior Tree (BT) and NAV2 BT Navigator	5
1.7	Action Server and Navigation Goals	5
1.8	TF2 and Robot Frames	5
1.9	Configuration and Customization	5
2	LABS	6
2.1	Lab 1: NAV2 System Observation	6
2.1.1	Requirements	6
2.1.2	Discussion	10
2.2	Lab 2: SLAM Toolbox and Mapping	11
2.2.1	Requirements	11
2.2.2	Discussion	12
2.3	Lab 3: Costmap and Inflation Layer Analysis	12
2.3.1	Requirements	12
2.3.2	Discussion	14
2.4	Lab 4: C++ Navigation Goal Sender	15
2.4.1	Requirements	15
2.5	Lab 5: Tuning Planners and Controllers	17
2.5.1	Requirements	17
2.5.2	Discussion	18
2.6	Lab 6: Behavior Tree Mechanics	18
3	Homework Assignments	20
3.1	Exercise 1: Patrol Robot	20
3.2	Exercise 2: Planner Metrics Comparison	23
3.3	Exercise 3: Custom Pure Pursuit Controller Plugin	23
4	References	26

1 Theoretical Background of NAV2

1.1 Overview of the Navigation Stack (NAV2)

NAV2 is the navigation framework for ROS2, enabling autonomous robots to move from point A to point B safely. Its core architecture comprises several communicating nodes, including global planners, local controllers, recovery behaviors, and a Behavior Tree (BT) navigator. NAV2 relies heavily on Lifecycle nodes to ensure the system boots up and shuts down deterministically, managing states such as unconfigured, inactive, and active.

1.2 Map and Localization

The environment is commonly represented using an Occupancy Grid Map, where data structures assign values to spatial grids: free space, occupied space, and unknown regions. SLAM (Simultaneous Localization and Mapping) is the computational problem of constructing a map of an unknown environment while simultaneously keeping track of an agent's location within it.

For localization within a pre-built map, AMCL (Adaptive Monte Carlo Localization) is utilized. It applies a particle filter algorithm to estimate the robot's pose. Key parameters include 'min_particles' and 'max_particles', which balance accuracy against computational load, and the laser model. The '/initialpose' topic is used to provide the filter with an initial estimate of the robot's coordinates.

1.3 Costmap

A 2D Costmap translates map and sensor data into a grid of cost values representing the difficulty or danger of traversing specific areas.

- **Global Costmap:** Operates over the entire static map, used primarily for long-distance path planning.
- **Local Costmap:** A smaller, rolling window around the robot, updating at high frequencies to facilitate real-time obstacle avoidance.

Costmaps consist of multiple layers: the 'StaticLayer' handles the pre-existing map, the 'ObstacleLayer' integrates real-time sensor data like LiDAR, and the 'InflationLayer' adds a safety buffer around known obstacles. Important tuning parameters include 'resolution', 'robot_radius', 'inflation_radius', and 'cost_scaling_factor'.

1.4 Path Planning

Path planning is divided into global and local tasks.

- **Global Planner:** Computes the optimal route from start to finish. Common algorithms include ‘NavFn’ (based on Dijkstra or A*), ‘Smac Planner 2D’, and ‘Theta* Planner’.
- **Local Planner (Controller):** Issues velocity commands to keep the robot on the global path while avoiding dynamic obstacles. Examples include ‘DWB’ (Dynamic Window Approach) and ‘RPP’ (Regulated Pure Pursuit), the latter being highly suitable for differential drive robots.

1.5 Recovery Behaviors

When a robot is stuck or a planner fails to find a valid path, NAV2 triggers recovery behaviors. Default behaviors include ‘ClearCostmapRecovery’ to purge stale sensor data, ‘SpinRecovery’ to clear space, ‘BackUpRecovery’ to reverse out of tight spots, and ‘WaitRecovery’. These are orchestrated by the Behavior Tree.

1.6 Behavior Tree (BT) and NAV2 BT Navigator

A Behavior Tree is a hierarchical structure dictating decision-making flow. It utilizes nodes such as Sequence, Fallback, Decorator, Action, and Condition to coordinate the navigation pipeline. NAV2 uses BTs to seamlessly link planning, control, and recovery. The XML structures defining these trees can be edited and visualized using tools like ‘Groot2’.

1.7 Action Server and Navigation Goals

In C++, ROS2 Actions handle long-running tasks using the ‘rclcpp_action’ interface. The ‘/navigate_to_pose’ action receives a target pose, provides continuous feedback, and returns a final result. Developers can write action clients with ‘rclcpp_action::create_client’ and handle asynchronous communication via goal response, feedback, and result callbacks.

1.8 TF2 and Robot Frames

NAV2 relies on the standard TF2 frame hierarchy: ‘map -> odom -> base_link -> sensor_frames’. The ‘map’ frame represents the fixed global origin, while the ‘odom’ frame provides continuous local tracking but suffers from cumulative drift over time. AMCL computes the transform between ‘map’ and ‘odom’ to correct this drift.

1.9 Configuration and Customization

NAV2 parameters are centralized in a YAML configuration file, typically loaded via ‘bringup_launch.py’. Planners and controllers are implemented as plugins via ROS2’s

‘pluginlib’, allowing developers to register and swap custom C++ algorithms dynamically.

2 LABS

2.1 Lab 1: NAV2 System Observation

2.1.1 Requirements

1. Based on the `ros2 node list` provided, the running system includes the core components of the Navigation 2 (Nav2) stack, simulation nodes (Gazebo), and robot-specific hardware drivers (TurtleBot3).

Nav2 Stack Nodes (Core Navigation):

- `/amcl`: Implements the Adaptive Monte Carlo Localization for robot positioning.
- `/map_server`: Loads and distributes the static map of the environment.
- `/planner_server`: Responsible for computing the global path from start to goal.
- `/controller_server`: Generates local velocity commands to follow the path and avoid obstacles.
- `/behavior_server`: Executes recovery behaviors like spinning or backing up when the robot is stuck.
- `/bt_navigator`: The orchestrator that manages navigation logic via Behavior Trees.
- `/waypoint_follower`: Handles the logic for visiting multiple target poses in sequence.
- `/smoother_server` & `/velocity_smoother`: Ensure that both the geometric path and the velocity profile are continuous and smooth.
- `/lifecycle_manager_localization` & `/lifecycle_manager_navigation`: Coordinate the operational states of the navigation modules.
- `/global_costmap/global_costmap` & `/local_costmap/local_costmap`: Maintain 2D grids representing obstacle density for path planning.
- `/nav2_container`: A container node for loading Nav2 components as shared library plugins.

2. Based on the provided `ros2 topic info` outputs, here is the analysis of the main navigation topics:

- `/scan`: LiDAR data.

- **Type:** sensor_msgs/msg/LaserScan
 - **Publisher count:** 1
 - **Subscription count:** 4
- /odom: Odometry data.
 - **Type:** nav_msgs/msg/Odometry
 - **Publisher count:** 1
 - **Subscription count:** 2
- /cmd_vel: Velocity commands.
 - **Type:** geometry_msgs/msg/Twist
 - **Publisher count:** 5
 - **Subscription count:** 1
- /map: Map data.
 - **Type:** nav_msgs/msg/OccupancyGrid
 - **Publisher count:** 1
 - **Subscription count:** 3
- /amcl_pose: Estimated pose from AMCL.
 - **Type:** geometry_msgs/msg/PoseWithCovarianceStamped
 - **Publisher count:** 1
 - **Subscription count:** 0

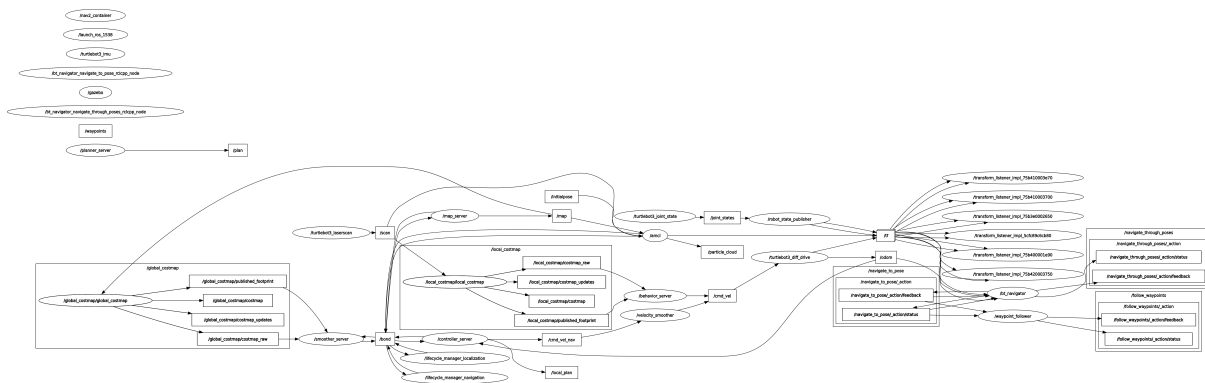
3. Based on the `ros2 action list` output, the following actions are currently running in the system:

- /backup
- /compute_path_through_poses
- /compute_path_to_pose
- /drive_on_heading
- /follow_path
- /follow_waypoints
- /navigate_through_poses
- /navigate_to_pose
- /smooth_path
- /spin
- /wait

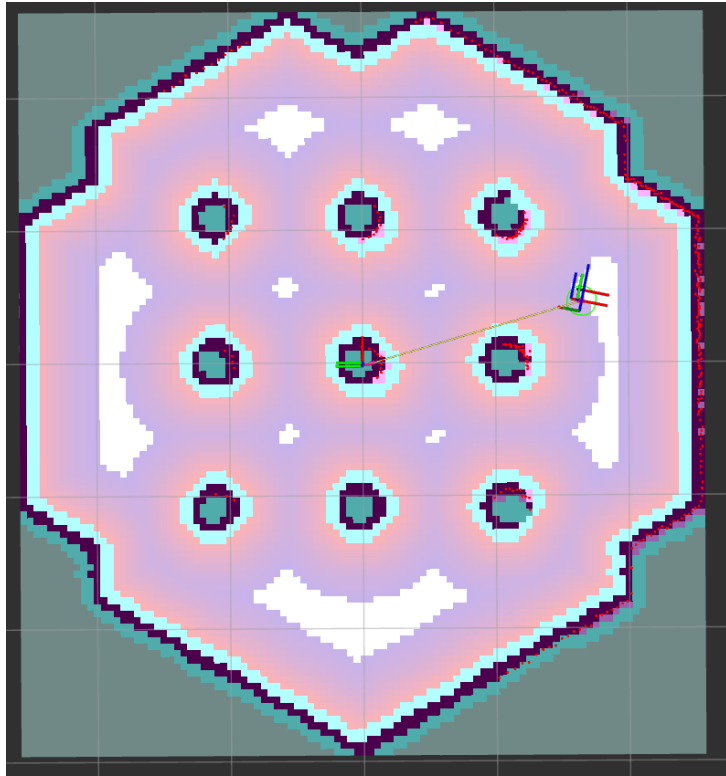
Based on the `ros2 action info /navigate_to_pose` output, here is the analysis of the navigation action:

- /navigate_to_pose
 - Action clients (4):
 - * /bt_navigator
 - * /waypoint_follower
 - * /rviz2
 - * /rviz_navigation_dialog_action_client
 - Action servers (1):
 - * /bt_navigator

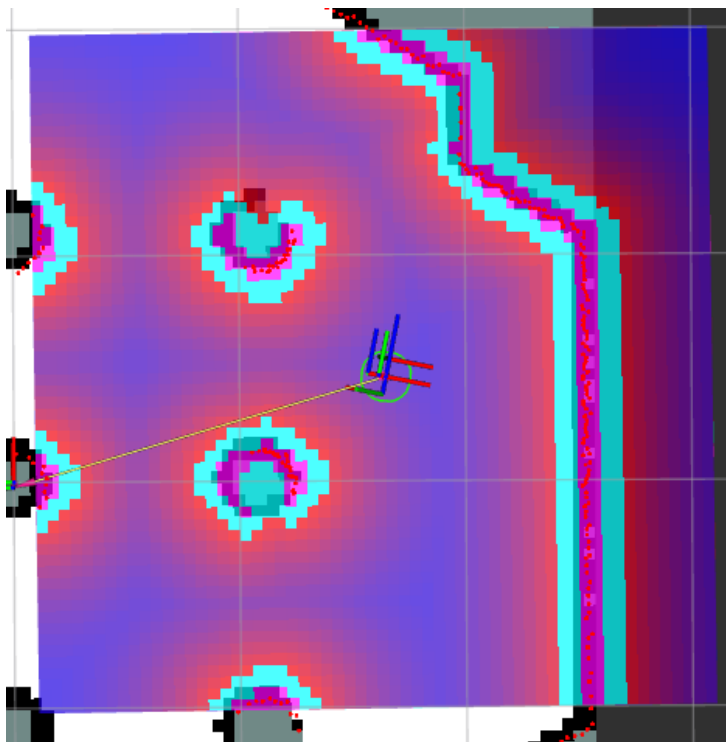
4. The `rqt_graph`:



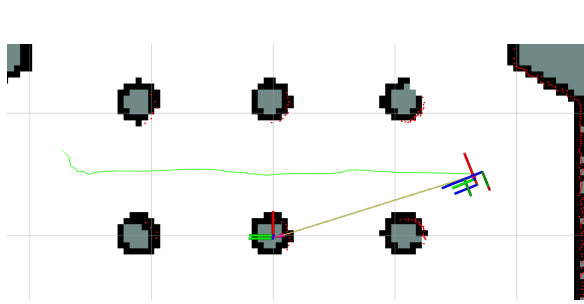
5. Global Costmap, Local Costmap, Path, Particle Cloud (AMCL)



Global Costmap



Local Costmap



Path (/plan topic)

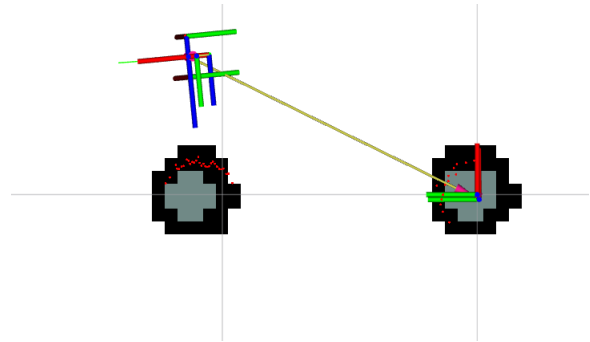
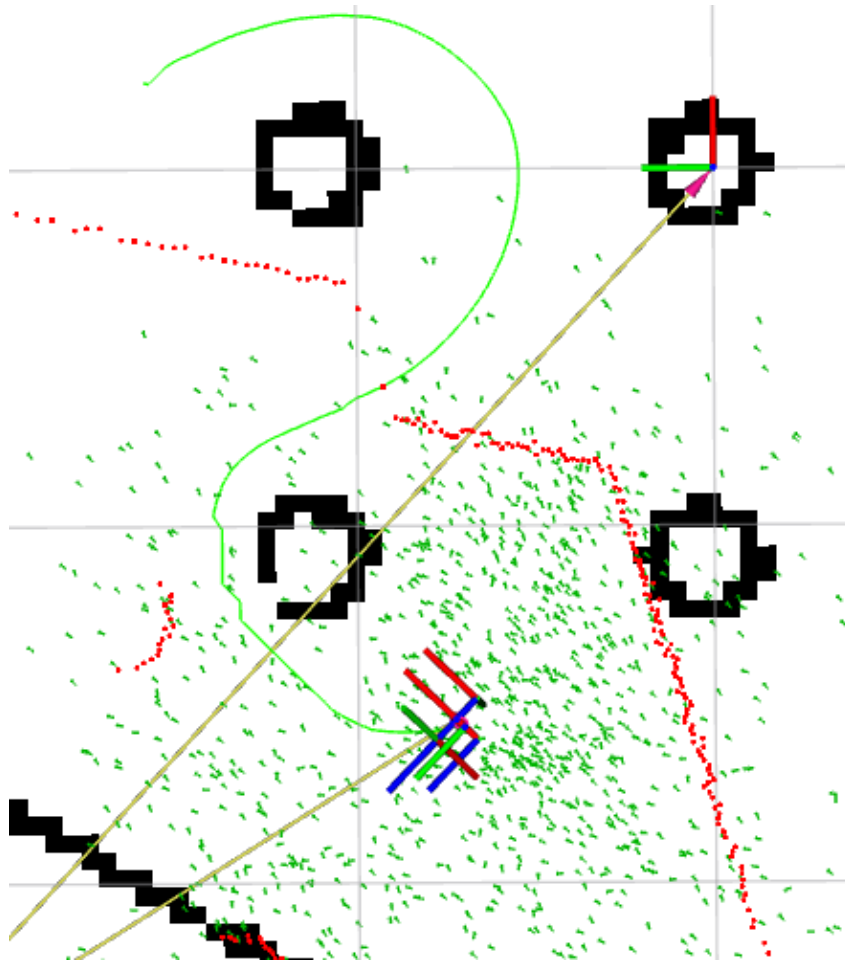


Figure 1: Path (/local_plan topic)



Particle Cloud (AMCL)

2.1.2 Discussion

- **Velocity Commands:** The `/cmd_vel` topic is published by the `controller_server` (the local planner) and subscribed to by the underlying hardware abstraction or simulation plugins (e.g., Gazebo differential drive plugin).
- **Localization Frames:** The `/amcl_pose` provides an absolute position estimate within the `map` frame, subject to discrete corrections. In contrast, the `/odom`

topic resides in the 'odom' frame, providing continuous, smooth measurements that inherently accumulate drift.

- **Costmap Duality:** Two separate costmaps exist to optimize performance. The Global Costmap covers the entire environment for long-term planning with low update frequency. The Local Costmap is a small, centered window updating rapidly to allow the controller to react to sudden, dynamic obstacles.

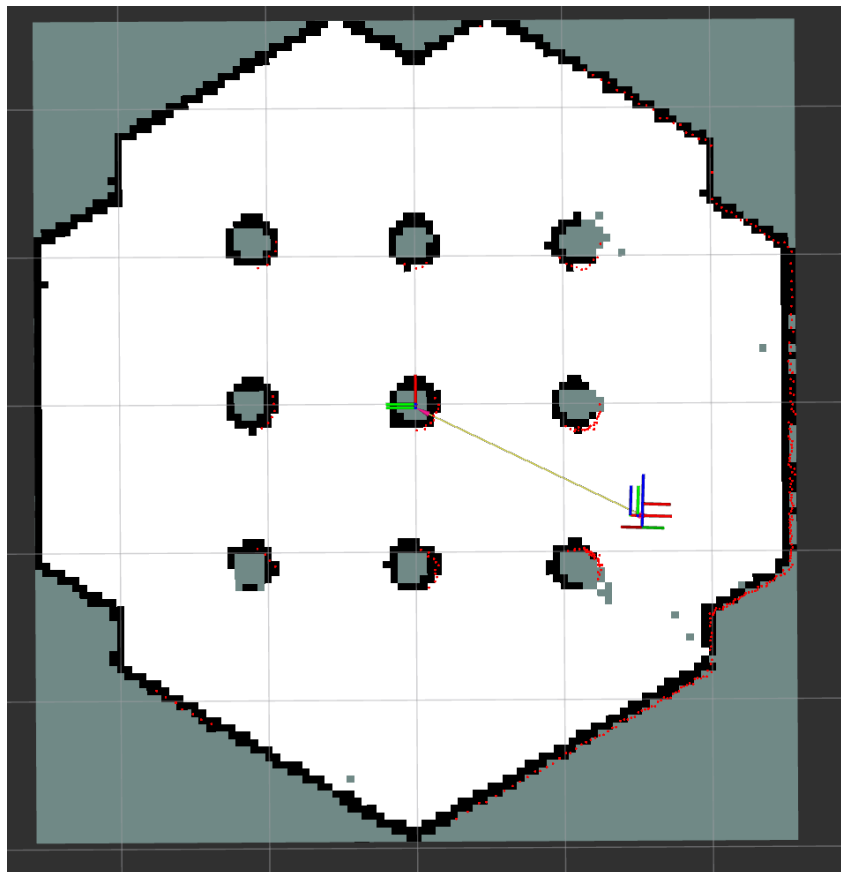
2.2 Lab 2: SLAM Toolbox and Mapping

2.2.1 Requirements

1. Scan map using Lidar

The video of the scanning using Lidar is uploaded [here](#).

2. The completed map:



Completed map (topic /map)

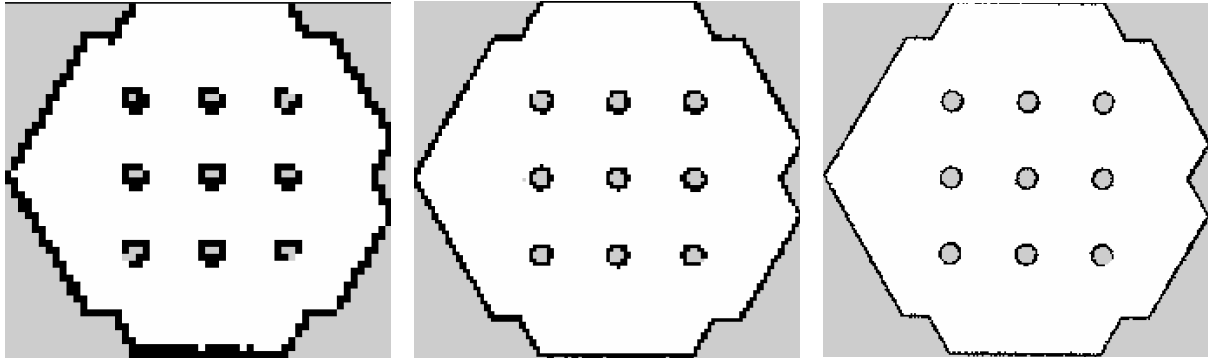
3. PGM & YAML:

- **PGM File:** Contains pixel values defining cell states: 0 represents occupied, 254 represents free space, and 205 represents unknown space.

- **YAML File:** Contains metadata mapping the pixels to real-world coordinates via parameters like ‘resolution’, ‘origin’, and occupancy thresholds.

4. Performance of SLAM Toolbox by varying the `resolution` parameter

5. Analysis on `resolution` parameter



Resolution: 0.1 m

Resolution: 0.05 m

Resolution: 0.025 m

- **0.1 m/pixel:** The map appears pixelated. Small obstacles like thin poles or sharp corners are blurred, which may lead to collisions during navigation because the robot’s footprint cannot be precisely calculated.
- **0.05 m/pixel (Default):** Provides a balanced trade-off. The structure of the environment is clearly defined while maintaining a manageable file size and computational load.
- **0.025 m/pixel:** The map is highly detailed, capturing fine geometric features. However, the memory footprint of the map doubles, and path planning algorithms (such as Nav2) require more CPU time to compute trajectories.

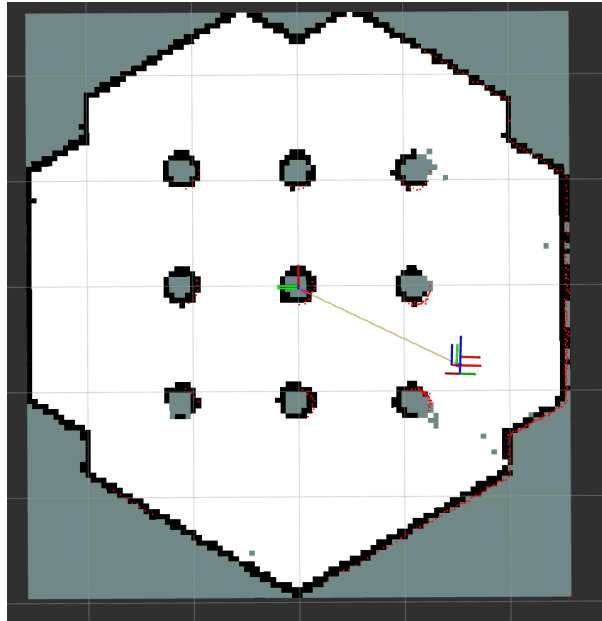
2.2.2 Discussion

The ‘resolution’ parameter heavily impacts system memory; a lower value (e.g., 0.025 m/pixel) creates a finer grid, increasing computational and memory demands but improving navigation accuracy around tight corners. If the robot is driven too rapidly or spun quickly during mapping, the LiDAR scans may blur, and odometry drift increases, causing SLAM algorithms to fail at loop closure and generating a distorted map.

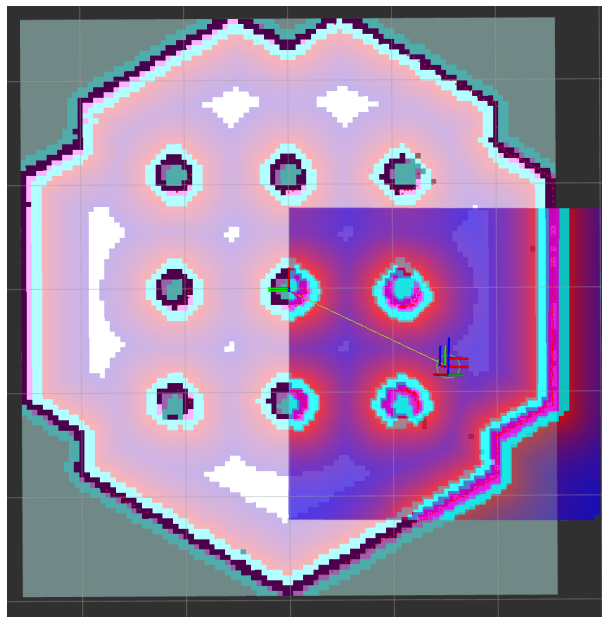
2.3 Lab 3: Costmap and Inflation Layer Analysis

2.3.1 Requirements

1. Map created at Lab 2:



2. Map (Global costmap and local costmap on)



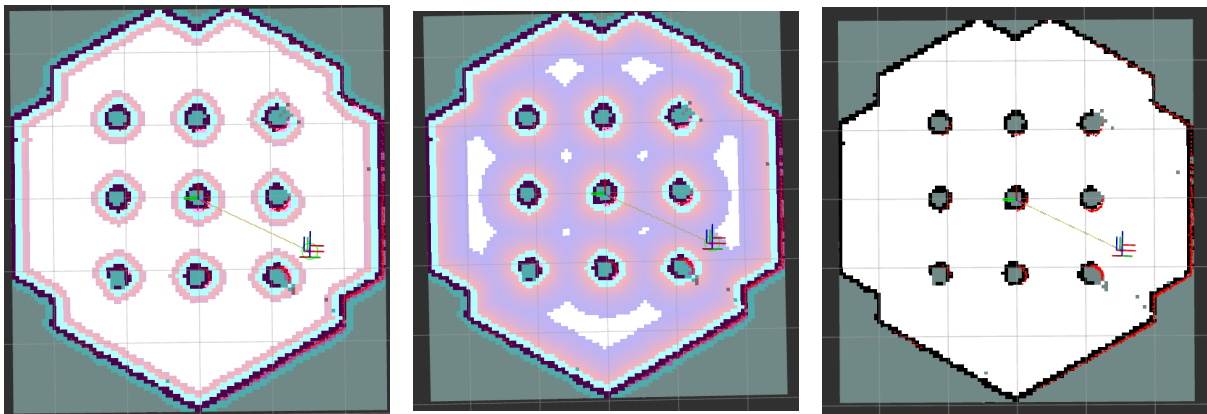
3. `my_nav2_params.yaml`

```

distance_waring > ! my_nav2_params.yaml
1  amcl:
2    ros_parameters:
3      alpha1: 0.2
4      alpha2: 0.2
5      alpha3: 0.2
6      alpha4: 0.2
7      alpha5: 0.2
8      base_frame_id: "base_footprint"
9      beam_skip_distance: 0.5
10     beam_skip_error_threshold: 0.9
11     beam_skip_threshold: 0.3
12     do_beamskip: false
13     global_frame_id: "map"
14     lambda_short: 0.1
15     laser_likelihood_max_dist: 2.0
16     laser_max_range: 100.0
17     laser_min_range: -1.0
18     laser_model_type: "likelihood_field"
19     max_beams: 60
20     max_particles: 2000
21     min_particles: 500
22     odom_frame_id: "odom"
23     pf_err: 0.05
24     pf_z: 0.99
25     recovery_alpha_fast: 0.0
26     recovery_alpha_slow: 0.0
27     resample_interval: 1
28     robot_model_type: "nav2_amcl::DifferentialMotionModel"
29     save_pose_rate: 0.5

```

4. inflation_radius adjustment



Global inflation_radius: 0.2 Global inflation_radius: 0.55 Global inflation_radius: 1.0

- inflation_radius: 0.2: The image clearly shows a very thin colored inflation layer around the pillars and walls. This perfectly aligns with the very thin visual size and implies the robot could get very close to obstacles.
- inflation_radius: 0.55: The colored gradient expands further out, providing a visible buffer. This matches the moderate visual size and the safe distance behavior.
- inflation_radius: 1.0: The third image shows almost no inflation gradient at all, meaning the whole map is overlapped with the inflation filter. This would likely lead to the path being blocked and the pathfinding status will also be blocked.

2.3.2 Discussion

The parameter ‘cost_scaling_factor’ controls the mathematical gradient of the inflation layer. A higher scaling factor causes the cost values to decay more rapidly as

the distance from the obstacle increases. If ‘inflation_radius’ is significantly larger than the physical width of a corridor in the map, the overlapping inflation boundaries will cause the entire corridor to possess a non-zero cost. If the combined cost reaches the lethal threshold, the global planner will completely refuse to route the robot through that corridor.

2.4 Lab 4: C++ Navigation Goal Sender

2.4.1 Requirements

The following is an overview of the Action Client implemented to send goals to ‘/navigate_to_pose’. The quaternion conversion is necessary because NAV2 requires orientations in quaternion format rather than Euler angles.

```

1 #include "rclcpp/rclcpp.hpp"
2 #include "rclcpp_action/rclcpp_action.hpp"
3 #include "nav2_msgs/action/navigate_to_pose.hpp"
4 #include "geometry_msgs/msg/pose_stamped.hpp"
5 #include <cmath>
6 #include "tf2/LinearMath/Quaternion.h"
7 using namespace std;
8
9 using NavigateToPose = nav2_msgs::action::NavigateToPose;
10 using GoalHandleNav = rclcpp_action::ClientGoalHandle<NavigateToPose>;
11 using namespace std::placeholders;
12
13 class GoalSenderNode : public rclcpp::Node
14 {
15 public:
16     GoalSenderNode() : Node("goal_sender")
17     {
18         client_ = rclcpp_action::create_client<NavigateToPose>(this, "
19         navigate_to_pose");
20     }
21
22     void send_goal(double x, double y, double yaw)
23     {
24         if (!client_->wait_for_action_server(std::chrono::seconds(5)))
25         {
26             RCLCPP_ERROR(get_logger(), "Action server not ready!");
27             return;
28         }
29
30         auto goal_msg = NavigateToPose::Goal();
31         goal_msg.pose.header.frame_id = "map";
32         goal_msg.pose.header.stamp = now();
33
34         goal_msg.pose.pose.position.x = x;

```

```

34     goal_msg.pose.pose.position.y = y;
35
36     // Chuyn i g c Yaw (radian) sang Quaternion
37     tf2::Quaternion q;
38     q.setRPY(0, 0, yaw);
39     goal_msg.pose.pose.orientation.x = q.x();
40     goal_msg.pose.pose.orientation.y = q.y();
41     goal_msg.pose.pose.orientation.z = q.z();
42     goal_msg.pose.pose.orientation.w = q.w();
43
44     auto send_goal_options = rclcpp_action::Client<NavigateToPose>::
SendGoalOptions();
45     // TODO: G n GoalResponseCallback, FeedbackCallback,
ResultCallback
46     send_goal_options.goal_response_callback = bind(&GoalSenderNode::
GoalResponseCallback, this, _1);
47     send_goal_options.feedback_callback = bind(&GoalSenderNode::
FeedbackCallback, this, _1, _2);
48     send_goal_options.result_callback = bind(&GoalSenderNode::
ResultCallback, this, _1);
49
50     client_ -> async_send_goal(goal_msg, send_goal_options);
51     RCLCPP_INFO(get_logger(), "Goal sent: (%.2f, %.2f, %.2f)", x, y,
yaw);
52 }
53
54
55 private:
56     rclcpp_action::Client<NavigateToPose>::SharedPtr client_;
57
58
59     void GoalResponseCallback(const GoalHandleNav::SharedPtr &
goal_handle)
60     {
61         // TODO
62         if (!goal_handle){
63             RCLCPP_ERROR(this -> get_logger(), "Goal Refused!");
64         }
65         else{
66             RCLCPP_INFO(this -> get_logger(), "Goal Accepted");
67         }
68     }
69
70     void FeedbackCallback(
71         GoalHandleNav::SharedPtr,
72         const std::shared_ptr<const NavigateToPose::Feedback> feedback)
73     {
74         // TODO: In v tr h i n t i v estimated_time_remaining

```

```

75 // 1 nh ros2 interface show <package_name/type/interface_name>
76 double x = feedback -> current_pose.pose.position.x;
77 double y = feedback -> current_pose.pose.position.y;
78 double z = feedback -> current_pose.pose.position.z;
79 // t u t h u c v o d n g m k h a i b o (
estimated_time_remaining c d n g sec + nanosec)
80 double t = feedback->estimated_time_remaining.sec + feedback->
estimated_time_remaining.nanosec / 1e9;
81
82 RCLCPP_INFO(this->get_logger(), "Current position: [%.2f, %.2f, %.2
f] \n Estimated time remaining: %.2f", x, y, z, t);
83 }
84
85 void ResultCallback(const GoalHandleNav::WrappedResult & result)
86 {
87 // TODO: X 1 SUCCEEDED, ABORTED, CANCELED
88 if (result.code == rclcpp_action::ResultCode::SUCCEEDED){
89 auto data = result.result->result;
90 RCLCPP_INFO(this->get_logger(), "Succeeded!");
91 }
92 else if (result.code == rclcpp_action::ResultCode::ABORTED){
93 RCLCPP_ERROR(this->get_logger(), "Aborted!");
94 }
95 else if (result.code == rclcpp_action::ResultCode::CANCELED){
96 RCLCPP_WARN(this->get_logger(), "Cancelled");
97 }
98 }
99 };
100
101 int main(int argc, char * argv[])
102 {
103 rclcpp::init(argc, argv);
104 auto node = std::make_shared<GoalSenderNode>();
105 node->send_goal(1.5, -0.5, 0.0);
106 rclcpp::spin(node);
107 rclcpp::shutdown();
108 return 0;
109 }

```

Listing 1: Completed Goal Sender C++ Implementation

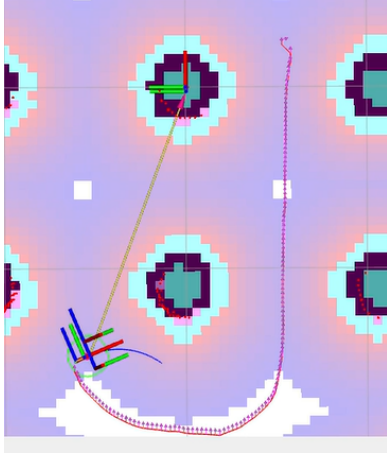
The video of the implementation of the `send_goal(1.5, -0.5, 0.0)`; in main is uploaded [here](#).

2.5 Lab 5: Tuning Planners and Controllers

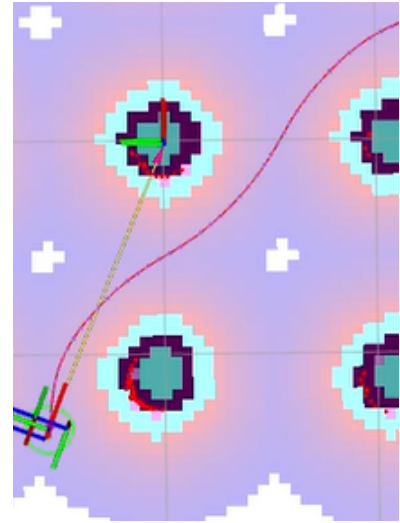
2.5.1 Requirements

Path planner of [NavFn global planner](#).

Path planner of [SmacPlanner2D](#) global planner.



Global NavFn global planner



Global SmacPlanner2D
global planner

- **Path shape:** SmacPlanner2D produces a smoother, more direct path compared to NavFn, which tends to generate paths that hug closer to obstacles due to its uniform cost expansion.
- **Path tracking behavior :** RPP tracks the path more smoothly with less oscillation by following a lookahead point, while DWB exhibits more frequent velocity adjustments as it continuously scores and selects from sampled trajectories.

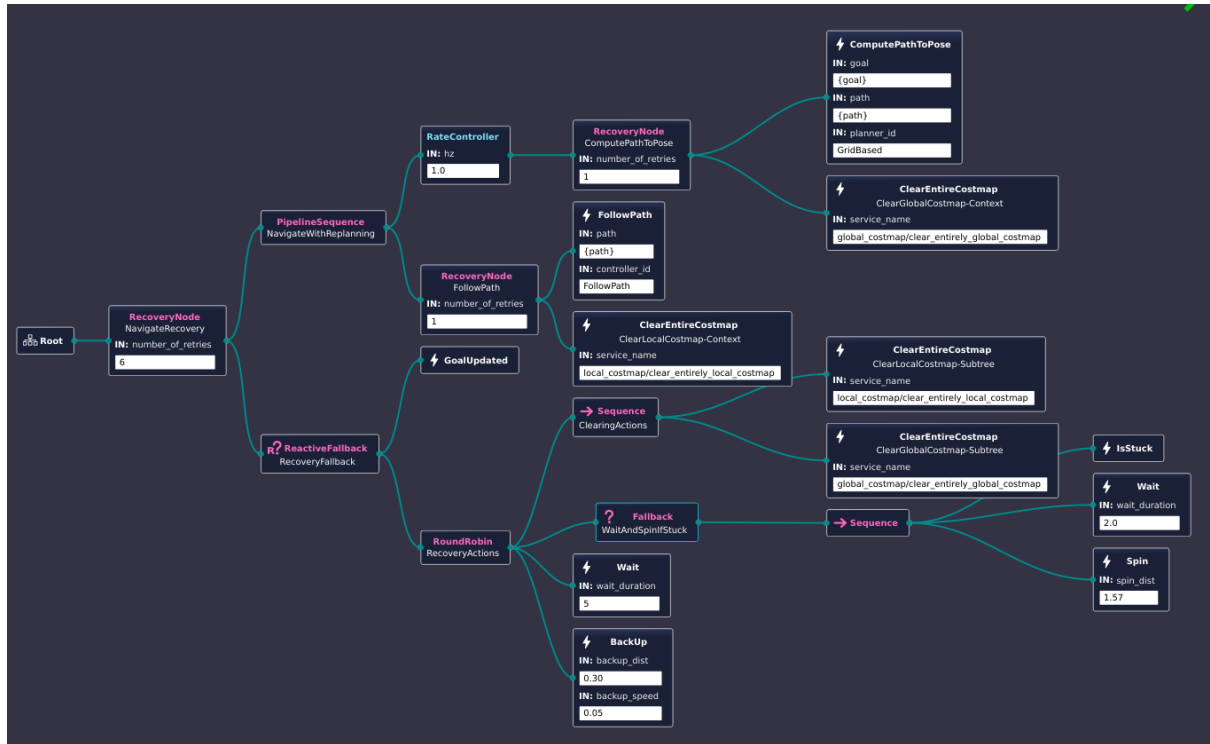
2.5.2 Discussion

Switching from the default ‘NavFn’ global planner to ‘SmacPlanner2D’ changes the underlying algorithm from Dijkstra to A*. While ‘NavFn’ uniformly floods the grid to find a path, ‘SmacPlanner2D’ utilizes Euclidean distance heuristics to bias the search toward the goal, resulting in faster computation times in expansive open spaces.

Transitioning the local controller from ‘DWB’ to ‘Regulated Pure Pursuit (RPP)’ alters tracking behavior. DWB simulates multiple velocity trajectories and scores them. RPP, conversely, identifies a lookahead point on the global path and generates smooth arcs towards it. For a differential drive robot like the TurtleBot3, RPP often yields smoother cornering and less oscillatory behavior compared to DWB.

2.6 Lab 6: Behavior Tree Mechanics

Inspecting the NAV2 Behavior Tree via Groot2 reveals its structure:



- ‘ComputePathToPose’ is the first child of a RecoveryNode (labeled ComputePathToPose). This node is responsible for calculating the global path from the current position to the destination. Wrapped in a RecoveryNode with number_of_retries="1", it will attempt to re-calculate once upon failure before reporting an error to the parent node.
- The RoundRobin node (labeled RecoveryActions) is positioned under a ReactiveFallback (labeled RecoveryFallback). It triggers whenever the main branch (PipelineSequence) returns FAILURE (e.g., the robot fails to plan a path or fails to execute the movement), executing one of its children at a time in a circular order each time it is ticked:
 1. On the first failure, it calls the Sequence (ClearingActions).
 2. On the second, it calls Spin.
 3. Then Wait, and finally BackUp. This allows the robot to cycle through different recovery behaviors rather than repeating the same failed action.
- The Sequence and Fallback nodes act as the decision-making "traffic lights":
 - PipelineSequence (Main Flow): This links path planning (ComputePathToPose) with execution (FollowPath). If either fails, the entire branch stops.
 - ReactiveFallback (Error Handling): Acts as a "safety net." If the navigation branch fails, it switches the flow to the RecoveryActions branch to attempt to resolve the issue.

- Sequence (ClearingActions): Used to group costmap clearing tasks (ClearEntireCostmap). It ensures that both Local and Global costmaps are cleared sequentially before considering the "cleanup" task successful.

3 Homework Assignments

3.1 Exercise 1: Patrol Robot

To achieve continuous patrolling behavior across four coordinates, an automated node must iterate through an array of targets, wait upon arrival, and handle failures gracefully.

```

1 #include <chrono>
2 #include <memory>
3 #include <tuple>
4 #include <vector>
5
6 #include "rclcpp/rclcpp.hpp"
7 #include "rclcpp_action/rclcpp_action.hpp"
8 #include "nav2_msgs/action/navigate_to_pose.hpp"
9 #include "tf2/LinearMath/Quaternion.h"
10 #include "tf2_geometry_msgs/tf2_geometry_msgs.hpp"
11 using namespace std;
12 using namespace chrono_literals;
13 using namespace placeholders;
14
15
16 class PatrolNode : public rclcpp::Node
17 {
18 public:
19     PatrolNode() : Node("patrol_node"), m_current_index(0){
20         m_nav_client = rclcpp_action::create_client<NavigateToPose>(this, "
            navigate_to_pose");
21
22
23         //4 waypoints on map (rviz2), use comm: ros2 topic echo /
            clicked_point
24         m_waypoints ={
25             {1.5228875875473022, -1.5027711391448975, -0.001434326171875},
26             {1.6150468587875366, 1.595870852470398, -0.005340576171875},
27             {-1.6925221681594849, 1.5239757299423218, 0.002471923828125},
28             {-1.8264621496200562, -1.3134897947311401, 0.002471923828125}
29         };
30
31         SendNextGoal();
32     }
33
34 private:

```

```

35 using NavigateToPose = nav2_msgs::action::NavigateToPose;
36 using GoalHandleNav = rclcpp_action::ClientGoalHandle<NavigateToPose
    >;
37
38
39 rclcpp_action::Client<NavigateToPose>::SharedPtr m_nav_client;
40
41 // Danh sách waypoint: {x, y, yaw}
42 std::vector<std::tuple<double, double, double>> m_waypoints;
43 size_t m_current_index{0};
44
45 rclcpp::TimerBase::SharedPtr m_timer;
46
47 void SendNextGoal(){
48     if (!m_nav_client -> wait_for_action_server(5s)){
49         RCLCPP_ERROR(get_logger(), "Nav action server not ready!");
50         return;
51     }
52     auto goal_msg = NavigateToPose::Goal();
53     goal_msg.pose.header.frame_id = "map";
54     goal_msg.pose.header.stamp = this->get_clock()->now();
55
56     //present coords
57     double x, y, yaw;
58     tie(x,y,yaw) = m_waypoints[m_current_index]; //tie = tuple
59
60     goal_msg.pose.pose.position.x = x;
61     goal_msg.pose.pose.position.y = y;
62
63     // Chuyển đổi góc Yaw (radian) sang Quaternion
64     tf2::Quaternion q;
65     q.setRPY(0, 0, yaw);
66     goal_msg.pose.pose.orientation.x = q.x();
67     goal_msg.pose.pose.orientation.y = q.y();
68     goal_msg.pose.pose.orientation.z = q.z();
69     goal_msg.pose.pose.orientation.w = q.w();
70
71
72     //start to move
73     RCLCPP_INFO(this->get_logger(), "[Start] Heading to waypoint %zu: (
x = %.2f, y = %.2f, z = %.2f)", m_current_index, x,y,yaw);
74
75     //callback
76     auto send_goal_options = rclcpp_action::Client<NavigateToPose>::
SendGoalOptions();
77     send_goal_options.result_callback = std::bind(&PatrolNode::
ResultCallback, this, _1);
78

```

```

79
80     m_nav_client -> async_send_goal(goal_msg, send_goal_options);
81 }
82 void ResultCallback(const GoalHandleNav::WrappedResult & result){
83     switch (result.code){
84         case rclcpp_action::ResultCode::SUCCEEDED:
85             RCLCPP_INFO(this->get_logger(), "[SUCCEEDED] Moved to
86 waypoint %zu", m_current_index);
87             break;
88         case rclcpp_action::ResultCode::ABORTED:
89             RCLCPP_WARN(this->get_logger(), "[ABORTED] Goal aborted at
90 waypoint %zu, moving to next waypoint", m_current_index);
91         case rclcpp_action::ResultCode::CANCELED:
92             RCLCPP_WARN(this->get_logger(), "[CANCELLED] Goal is
93 cancelled.");
94             return;
95         default:
96             RCLCPP_ERROR(this->get_logger(), "[ERROR] Unknown.");
97             break;
98     }
99     m_current_index = (m_current_index+1)%m_waypoints.size();
100     m_timer = this->create_wall_timer(
101         3s, bind(&PatrolNode::SendNextGoal, this));
102 }
103 };
104
105 void graceful_shutdown(){
106     RCLCPP_INFO(rclcpp::get_logger("patrol_node"), "Control C ->
107 Shutting down...");
108 }
109
110 int main(int argc, char ** argv)
111 {
112     rclcpp::init(argc, argv);
113
114     // ng k h m x l shutdown
115     rclcpp::on_shutdown(graceful_shutdown);
116
117     auto node = std::make_shared<PatrolNode>();
118
119     rclcpp::spin(node);
120
121     rclcpp::shutdown();
122     return 0;
123 }

```

Listing 2: Patrol Node Core Logic

The simulation video is upload [here](#).

3.2 Exercise 2: Planner Metrics Comparison

When analyzing ‘NavFn’ and ‘SmacPlanner2D’, the accumulated distance is calculated using consecutive ‘/odom’ messages via the formula $D_{total} = \sum \sqrt{\Delta x^2 + \Delta y^2}$. Experimental metrics generally show that while ‘NavFn’ creates slightly safer trajectories centered between obstacles, ‘SmacPlanner2D’ computes the initial path significantly faster due to heuristic guidance.

Table 1: Experimental Results Comparison

Planner	Run	Time (s)	Distance (m)	Recovery Triggered
NavFn	1	20.13	3.55	No
NavFn	2	20.86	3.28	No
NavFn	3	21.71	3.65	No
NavFn	4	19.56	3.70	No
NavFn	5	19.78	3.86	No
Average	20.41 ± 0.70	$3.61 \pm 0.23 \pm$		
SmacPlanner2D	1	19.88	3.47	No
SmacPlanner2D	2	20.41	3.14	No
SmacPlanner2D	3	20.26	3.56	No
SmacPlanner2D	4	19.38	3.46	No
SmacPlanner2D	5	21.03	3.90	No
Average	20.19 ± 0.55	3.51 ± 0.25		

The simulation video of the NavFn planner is uploaded [here](#).

3.3 Exercise 3: Custom Pure Pursuit Controller Plugin

A custom ‘Controller’ plugin requires calculating angular velocity based on geometric Pure Pursuit. Given a constant linear velocity v , a lookahead distance L_d , and the angular deviation to the lookahead point α , the angular velocity ω is calculated as:

$$\omega = \frac{2v \sin(\alpha)}{L_d}$$

```
1 #include "my_pure_pursuit/pure_pursuit_controller.hpp"
2 #include "nav2_util/node_utils.hpp"
3 #include "tf2_geometry_msgs/tf2_geometry_msgs.hpp"
4
```

```

5 namespace my_pure_pursuit {
6
7 void PurePursuitController::configure(
8     const rclcpp_lifecycle::LifecycleNode::WeakPtr & parent,
9     std::string name, std::shared_ptr<tf2_ros::Buffer> tf,
10    std::shared_ptr<nav2_costmap_2d::Costmap2DROS> /*costmap_ros*/)
11 {
12     m_tf = tf;
13     auto node = parent.lock();
14
15     m_clock = node->get_clock();
16
17     // Khai báo tham số costmap trong file YAML
18     nav2_util::declare_parameter_if_not_declared(node, name + ".
19         lookahead_dist", rclcpp::ParameterValue(0.5));
20     node->get_parameter(name + ".lookahead_dist", m_lookahead_dist);
21
22     RCLCPP_INFO(m_logger, "PurePursuitController configured!");
23 }
24
25 void PurePursuitController::setPlan(const nav_msgs::msg::Path & path) {
26     m_current_plan = path;
27 }
28
29 geometry_msgs::msg::TwistStamped PurePursuitController::
30     computeVelocityCommands(
31         const geometry_msgs::msg::PoseStamped & /*pose*/,
32         const geometry_msgs::msg::Twist & /*velocity*/,
33         nav2_core::GoalChecker * /*goal_checker*/)
34 {
35     geometry_msgs::msg::TwistStamped cmd;
36     cmd.header.frame_id = "base_link";
37     cmd.header.stamp = m_clock->now();
38
39     if (m_current_plan.poses.empty()) return cmd;
40
41     std::string path_frame = m_current_plan.header.frame_id;
42     if (path_frame.empty()) path_frame = "map";
43
44     geometry_msgs::msg::TransformStamped transform;
45     transform = m_tf->lookupTransform("base_link", path_frame, tf2::
46         TimePointZero);
47
48     // --- PHẦN CODE MÃI : T NH K H O N G C C H N C H C U I
49     C N G ---
50
51     geometry_msgs::msg::PoseStamped final_pose = m_current_plan.poses.
52         back();

```

```

48 geometry_msgs::msg::PoseStamped local_final_pose;
49 tf2::doTransform(final_pose, local_final_pose, transform);
50
51 double dist_to_goal = std::hypot(local_final_pose.pose.position.x,
52     local_final_pose.pose.position.y);
53
54 // 1. Nếu twist gần ch (c ch < 0.1 m t), d ng
55     xe ngay l p t c !
56 if (dist_to_goal < 0.1) {
57     cmd.twist.linear.x = 0.0;
58     cmd.twist.angular.z = 0.0;
59     RCLCPP_INFO(m_logger, "Reached goal.");
60     return cmd;
61 }
62
63 // 2. Tính toán vận tốc (Gim t c khi c ch ch n h
64     h n t m nh n lookahead_dist)
65 double current_lin_vel = m_linear_vel;
66 if (dist_to_goal < m_lookahead_dist) {
67     // p t c gim dn theo t l khong c ch
68     current_lin_vel = m_linear_vel * (dist_to_goal / m_lookahead_dist);
69
70     // Gi m t m c ga t i thiu (0.05) xe kh ng b "
71     c h t ng " tr c khi c h m ch
72     if (current_lin_vel < 0.05) current_lin_vel = 0.05;
73 }
74 // -----
75
76 geometry_msgs::msg::PoseStamped local_target;
77 bool found_pt = false;
78
79 for (const auto & p : m_current_plan.poses) {
80     geometry_msgs::msg::PoseStamped p_with_frame = p;
81     p_with_frame.header.frame_id = path_frame;
82
83     geometry_msgs::msg::PoseStamped local_p;
84     tf2::doTransform(p_with_frame, local_p, transform);
85
86     double dist = std::hypot(local_p.pose.position.x, local_p.pose.
87         position.y);
88
89     if (dist >= m_lookahead_dist && local_p.pose.position.x > 0.0) {
90         local_target = local_p;
91         found_pt = true;
92         break;
93     }
94 }
95 }

```

```

91  if (!found_pt) {
92      local_target = local_final_pose; // T n   d n g   l u n   b i n
          local_final_pose           t n h           t r n
93  }
94
95  double x = local_target.pose.position.x;
96  double y = local_target.pose.position.y;
97  double L2 = x*x + y*y;
98
99  double curvature = (L2 > 0.001) ? (2.0 * y) / L2 : 0.0;
100
101  // 3. S   d n g   c u r r e n t _ l i n _ v e l (   c   t n h   n n g   g i m   t c )
          t h a y   v   m _ l i n e a r _ v e l   c           n h
102  cmd.twist.linear.x = current_lin_vel;
103  cmd.twist.angular.z = current_lin_vel * curvature;
104
105  // RCLCPP_INFO(m_logger, "Dist to goal: %.2f | Vel: %.2f | Ang: %.2f
          ", dist_to_goal, cmd.twist.linear.x, cmd.twist.angular.z);
106
107  return cmd;
108 }
109
110
111 void PurePursuitController::setSpeedLimit(const double & /*speed_limit
          */, const bool & /*percentage*/) {}
112
113 } // namespace
114
115 #include "pluginlib/class_list_macros.hpp"
116 PLUGINLIB_EXPORT_CLASS(my_pure_pursuit::PurePursuitController,
          nav2_core::Controller)

```

Listing 3: Compute Velocity Commands (Pure Pursuit)

The simulation is uploaded [here](#).

4 References

References

- [1] NAV2 Development Team, “NAV2 Documentation (Official),” . Available: <https://docs.nav2.org>.
- [2] Open Navigation, “NAV2 GitHub Repository”. Available: <https://github.com/ros-navigation/navigation2>.
- [3] Open Robotics, “ROS2 Documentation”. Available: <https://docs.ros.org>.

- [4] ROBOTIS, “TurtleBot3 e-Manual”. Available: <https://emanual.robotis.com/docs/en/platform/turt>
- [5] Open Robotics, “rclcpp_action API Reference”. Available: https://docs.ros2.org/latest/api/rclcpp_action.
- [6] The Construct, “NAV2 Tutorial Series”. Available: [YouTube](#).
- [7] Articulated Robotics, “NAV2 Playlist”. Available: [YouTube](#).
- [8] Robotics Back-End, “ROS2 Navigation Tutorials”. Available: [YouTube](#).
- [9] S. Macenski, “slam_toolbox: A flexible and fast SLAM library for ROS2”. Available: [GitHub](#).
- [10] BehaviorTree.CPP, “BehaviorTree.CPP Library,”. Available: [GitHub](#).
- [11] BehaviorTree.CPP, “Groot2: Behavior Tree Visualizer and Editor,” .
- [12] ROS Wiki, “pluginlib: A C++ library for loading and unloading plugins”.
- [13] LTC21 Tutorial Pure Pursuit Available: [YouTube](#).